

Technical Report Deliverable #3

Use Case: Dynamic Grade Exporter

Name

Strategy Pattern

What it does

The Strategy Pattern allows a class to perform a task in different ways by "swapping out" the logic at runtime. Instead of having one giant function filled with if-else or switch statements to decide how to export a file, you create separate "strategies" for each format (CSV, PDF, Text). The main class simply tells the current strategy to "do the work," without needing to know *how* that specific strategy functions.

Why we used this pattern

We used this pattern for three main reasons:

1. **Cleaner Code (No "If-Else")**: It removes complex conditional blocks from the core GradeExporter class, making it much easier to read and maintain.
2. **Easy to Expand**: If you decide to add a new export format—like JSON—you don't have to change any of your existing code. You simply create a new class (e.g., JSONGradeExportStrategy) that follows the existing interface.
3. **Flexibility**: It allows the application to change behavior while it is running. The user can switch from a CSV export to a PDF export instantly, and the system handles the transition seamlessly.

Use Case: Dynamic Notification System

Name

Strategy Pattern

What it does

The Strategy Pattern acts like a "universal remote" for notification delivery. Instead of the NotificationSender having to know the technical details of how to send an SMS, an Email, or an In-App message, it simply holds a reference to a generic NotificationDeliveryStrategy.

When the system needs to send an alert, it doesn't care *which* channel is being used; it just calls the send method. Because we have separate classes for each channel, the system can seamlessly switch between them at runtime—for example, sending urgent alerts via SMS while sending routine updates via Email.

Why we used this pattern

We used the Strategy pattern here for three key benefits:

1. **Strict Adherence to OCP (Open/Closed Principle):** Our system is now "closed" to modification—you don't have to touch the NotificationSender code if you want to add a new delivery method. It is "open" to extension—you can add WhatsApp or Slack notifications simply by creating a new class that implements the NotificationDeliveryStrategy interface.
2. **Modular Maintenance:** If the SMS gateway changes (e.g., switching from Twilio to another provider), you only need to update the SMSNotificationStrategy class. The rest of your application remains completely untouched and stable.
3. **Encapsulation of Complexity:** The NotificationSender doesn't get cluttered with various API configurations or connection logic for different services. By isolating the logic for each notification channel into its own dedicated class, the code remains clean, readable, and easy to test.

Use Case: Announcement & Event Management System

Name

Observer + Singleton Pattern

What it does

This code uses two patterns working in tandem:

- **Singleton:** Ensures there is only **one** EventManager in the entire application. Every part of your program refers to the same "central hub" for events, so you don't have to pass an object around to every single panel.
- **Observer:** Allows different parts of the system (like UI panels or SMS services) to "subscribe" to specific events. When something happens (like a new announcement), the EventManager broadcasts the news, and all subscribed panels update themselves automatically.

Why we used this pattern

We combined these two patterns to solve a major communication problem:

1. **Eliminating Tight Coupling:** Without this, the Teacher or Database classes would need a list of every UI panel and notification service they need to update. If you added a new panel, you would have to change the core logic. Now, the EventManager handles the list of subscribers; the teacher just "broadcasts" the event, and the subscribers handle the rest.
2. **Centralized Control (Singleton):** By using a Singleton, we ensure that every panel is connecting to the *same* event stream. This prevents issues where one part of the app is "listening" to an event manager that isn't actually receiving the updates.
3. **Flexibility and Scalability:** This makes the system incredibly easy to grow. Want to add an "Email Notification Panel" or a "Parent Alert System"? You just create the class, implement the EventListener interface, and call subscribe(). You **never** have to change the EventManager or the code that generates the announcements.

4. **Selective Updates:** Panels only receive the notifications they actually care about (e.g., the AttendanceAlertListener only reacts to attendance, while the SMSAlertPanel might react to exams and announcements), making the system highly efficient.

Use Case: Course Creation Workflow

Name

Template Method Pattern

What it does

The Template Method pattern defines the "skeleton" of an algorithm in an abstract base class. It dictates the exact sequence of steps to be performed (Title → Content → Schedule → Publish). The base class makes these steps abstract, forcing the subclasses (OnlineCourse and OfflineCourse) to provide the specific implementation for each step.

The createCourse() method is marked final, which ensures that the overarching workflow cannot be changed by subclasses, maintaining consistency across the entire application.

Why we used this pattern

We used this pattern to enforce a rigid process while allowing flexibility in the details:

1. **Code Reuse:** All the shared logic (the sequence of steps) is located in one place (the CourseCreation class). You don't have to rewrite the workflow logic for every new type of course.
2. **Process Consistency:** Because the createCourse() method is final, you guarantee that every course creation follows the exact same required steps. No one can accidentally skip the "publish" step or change the order of operations.

3. **Encapsulation of Variation:** Each subclass only cares about its specific implementation details. The OfflineCourse class doesn't need to know how to upload videos, and the OnlineCourse doesn't need to know how to set classroom timings.
4. **Simplified Extension:** If you want to add a new type of course (e.g., HybridCourse), you simply extend the CourseCreation class and fill in the required steps. The rest of your system will instantly recognize and work with this new course type because it fits the defined "template."

Use Case: Attendance Marking Process

Name

Template Method Pattern

What it does

The Template Method pattern defines the high-level workflow of marking attendance (load → mark → validate → save) in the AttendanceProcess abstract class. By marking the main markAttendance() method as final, the pattern ensures that the core process remains identical regardless of the method being used. Subclasses like ManualAttendance and QRAttendance are then forced to "fill in the blanks" by implementing the specific details of how that attendance is recorded and validated.

Why we used this pattern

We implemented this pattern for the attendance system for several key reasons:

1. **Enforced Standard Procedure:** Attendance policies are usually strict. Using a final template method prevents developers from accidentally skipping the validate() or save() steps when adding a new attendance type (e.g., Biometric or GPS-based attendance).
2. **Increased Maintainability:** If the school changes the "save" procedure (e.g., migrating to a new database system), you only need to update the save()

implementation in the specific subclasses, or—if the save logic is shared—you could implement it once in the base class.

3. **Clean Separation of Roles:** The logic for "how" a student is marked (manual vs. QR) is decoupled from the "when" and "what" of the overall attendance workflow. This makes the code much easier to read because the high-level algorithm is visible at a glance in the abstract base class.
4. **Open/Closed Principle:** The system is "closed" to changes in the overall attendance workflow but "open" to adding new, creative ways of marking attendance. If you needed to add a "Facial Recognition" attendance method tomorrow, you would only need to create a new class and define those four specific steps.

Use Case: Student List Traversal

Name

Iterator Pattern

What it does

The Iterator Pattern provides a way to access the elements of a collection (like our `StudentCollection`) sequentially without exposing the underlying implementation (the fact that it uses an `ArrayList`).

By creating a separate `StudentIterator` class, we "hide" the internal structure of the data. The client code doesn't need to know if the students are stored in an `ArrayList`, a `LinkedList`, or even a custom database query; it simply asks the iterator for the `next()` item until `hasNext()` returns false.

Why we used this pattern

We used this pattern for the following reasons:

1. **Uniform Traversal:** It provides a standard way to loop through a collection. If you later decide to change how students are stored (e.g., changing from a

simple List to a complex tree structure), the client code that iterates through the students doesn't have to change at all.

2. **Single Responsibility Principle:** The StudentCollection is responsible for managing the student data, while the StudentListIterator is responsible for the traversal logic. This separation keeps the code clean and focused.
3. **Multiple Iterators:** You can have multiple, independent iterators running at the same time on the same collection. For example, one part of your app could be scanning students to send an email, while another part is scanning to generate a report, and they won't interfere with each other's position in the list.
4. **Simplified Client Code:** The client doesn't need to manage indices or worry about IndexOutOfBoundsException exceptions. It uses a clean, consistent interface (hasNext and next) that makes the code more readable and less prone to errors.

Use Case: Classroom Chat Room

Name

Mediator Pattern

What it does

The Mediator pattern acts as a central "hub" for communication. Instead of every student and teacher having to know about and hold a reference to every other participant in the chat (which would create a "spaghetti" of connections), they only talk to the ClassroomChatMediator. The mediator takes the message and decides who needs to receive it, keeping the participants completely decoupled from one another.

Why we used this pattern

We used the Mediator pattern here to solve the "Many-to-Many" communication problem:

1. **Reduced Complexity (Decoupling):** Without a mediator, if you had 30 students, each student would need a list of 29 others to send messages. If a new student joined, you would have to update every existing student's list. With the mediator, participants don't even know each other exist—they only know the mediator.
2. **Simplified Maintenance:** All the routing logic (e.g., "should the teacher see this?" or "can students send private messages?") is centralized in the ClassroomChatMediator. If you want to add a new feature, like filtering bad language, you only have to change it in one place.
3. **Encapsulation of Behavior:** The StudentUser and TeacherUser classes are focused only on sending and receiving. They aren't burdened with the complex logic of managing a list of users or broadcasting messages, making them much simpler to write and test.
4. **Reusability:** Because the participants only depend on the ChatMediator interface, you could easily reuse the same StudentUser class in a completely different environment (like an email system or a forum) without needing to modify the user's core logic.

Use Case: Exam Room Coordination

Name

Mediator Pattern

What it does

In this scenario, the ExamRoomMediator acts as the "control center" for the exam environment. The ExamTimer, StudentPanel, and ProctorPanel are independent components that don't know about each other. Instead of the timer having to manually "call" the student panel to lock it or the proctor panel to alert them, it simply sends a notification to the mediator. The mediator holds the logic for how these components should react to one another (e.g., when the event TIME_UP

happens, the mediator knows it must call `lock()` on the student panel *and* alert the proctor panel).

Why we used this pattern

We used the Mediator pattern here to manage complex interdependencies:

1. **Orchestration of Logic:** The exam process involves a sequence of events where one component's state change triggers actions in others. Placing this orchestration in a mediator prevents the components themselves from becoming bloated with logic that doesn't belong to them (e.g., the `ExamTimer` should only care about time, not about what the `ProctorPanel` does).
2. **Decoupling:** The `StudentPanel` is completely unaware of the `ProctorPanel`. You could change the proctoring system (e.g., to a remote AI-based proctoring system) by only updating the mediator, without ever touching the `StudentPanel` or `ExamTimer` code.
3. **Preventing Circular Dependencies:** In a complex system, you might have Component A needing Component B, and Component B needing Component A. A mediator resolves this by providing a single point of interaction, which keeps the architecture clean and prevents "spaghetti code."
4. **Easy Workflow Updates:** If the school changes the exam rules—for example, if they decide that "Submitting an Answer" should also stop the timer—you only modify the `notify` method in the `ExamRoomMediator`. The individual components remain exactly as they were.